Paper Review • arXiv 2610.42141

RoboArchitect

Automating the Design of Robotic Agents with Persistent Memory

Coding LLM auto-designs a frozen VLA's agent harness, changing no weights

Shengguang Wu, Suning Huang, Cherie Ho, Ria Doshi, Hao Zhu, Serena Yeung-Levy
Stanford University • First submitted 2026-10-30

In one sentence: editing only code outside the policy lifts held-out success 3–4.5×

Without updating any weights, a coding LLM watches robot rollout videos, diagnoses failures, and iteratively rewrites the harness around a frozen VLA (tool calls + programmatic control + self-designed persistent memory); **after 5 rounds, held-out success rises sharply,** often surpassing the same model's fine-tuned version outside the policy's training distribution.

9.7→43.3%

Unseen scenes
Avg. success • 4.5×

12.7→40.2%

Unseen tasks
Avg. success • 3.2×

0

Weight updates / demos
Policy stays frozen

<$150

One full evolution
5 rounds • <6 h of coding-model time

Covers 5 simulation benchmarks × 7 VLA policies; all scores are on held-out episodes unseen during optimization.

Motivation: harnesses help, but are still hand-built and redone per robot



↑ Manual harness trial-and-error: slow, videos hard to inspect frame by frame, redone for each robot/benchmark

Usual path to new settings: fine-tuning

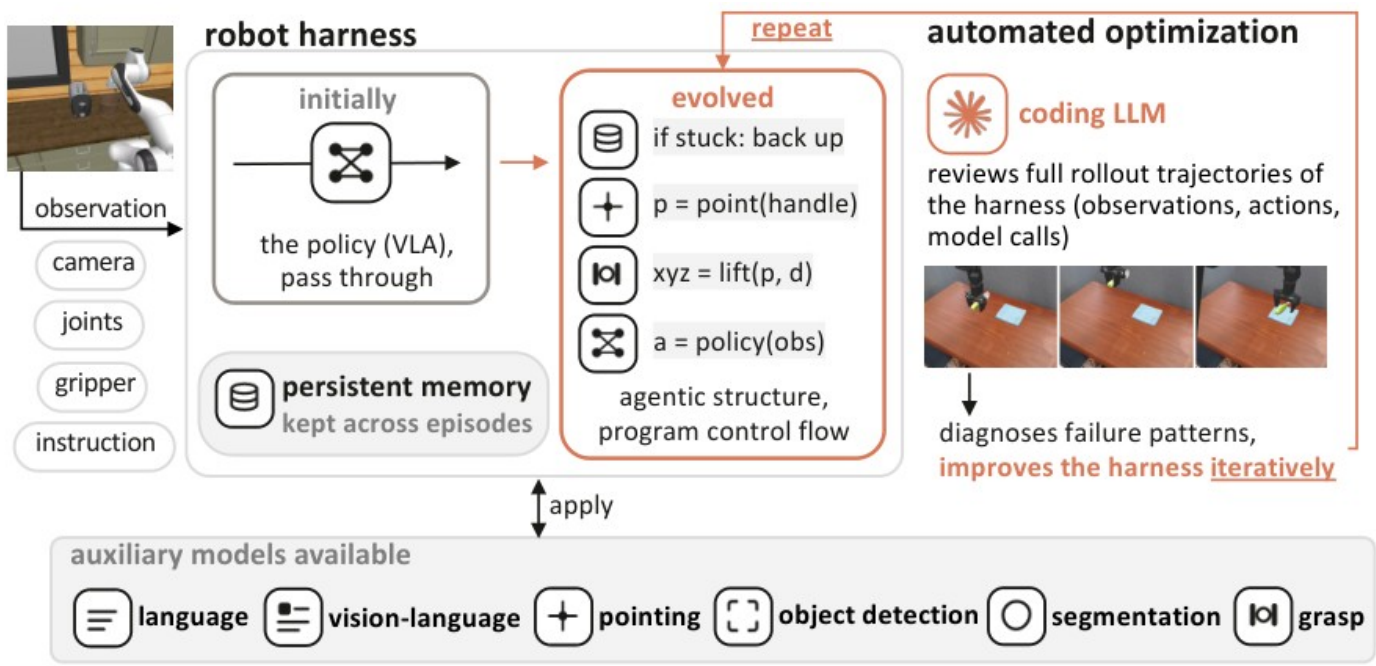
- Collect demos for each new task / hardware
- Update model weights (OpenVLA, π0.5, etc.)
- Fine-tuned skills often fail to generalize beyond training distribution (LIBERO-Pro)
- Inference-time methods for frozen policies (demo retrieval, value-based action selection) use fixed, predefined mechanisms

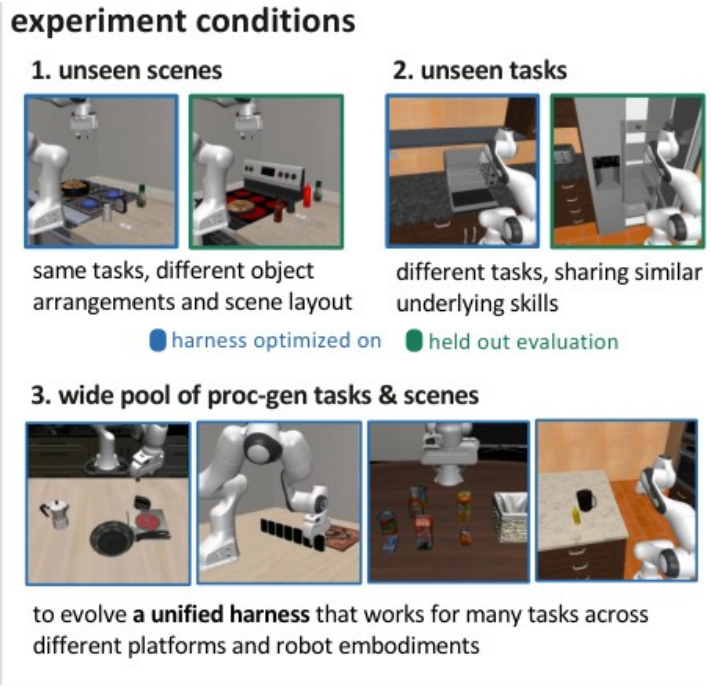
This paper asks

Can a coding LLM automate the full loop of "inspect failed rollouts → revise harness"?

- Freeze the policy; optimize only outer code
- Every edit is readable code: inspectable and iterable
- Data decides policy/code division and what memory stores

Method: a coding model reads rollouts, rewrites the harness, keeps only train-set gains





Harness interface

Camera/depth/gripper/ instruction in, low-level actions out; initialized as VLA passthrough (bare policy)

Auxiliary tools

Qwen3 LLM, Molmo2/Qwen3-VL, Grounding-DINO, SAM 3, GraspGen + geometric tools (e.g., 3D projection)

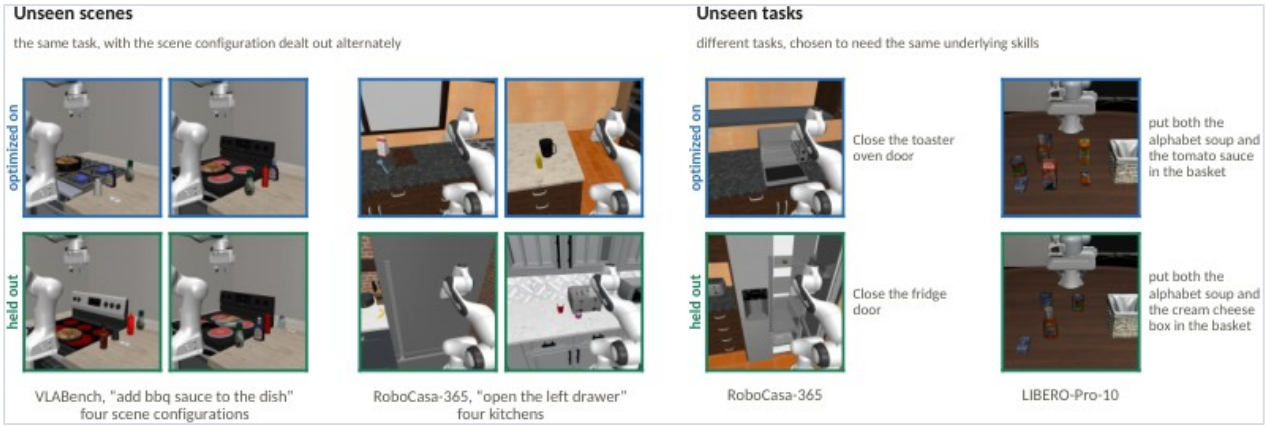
Optimization loop

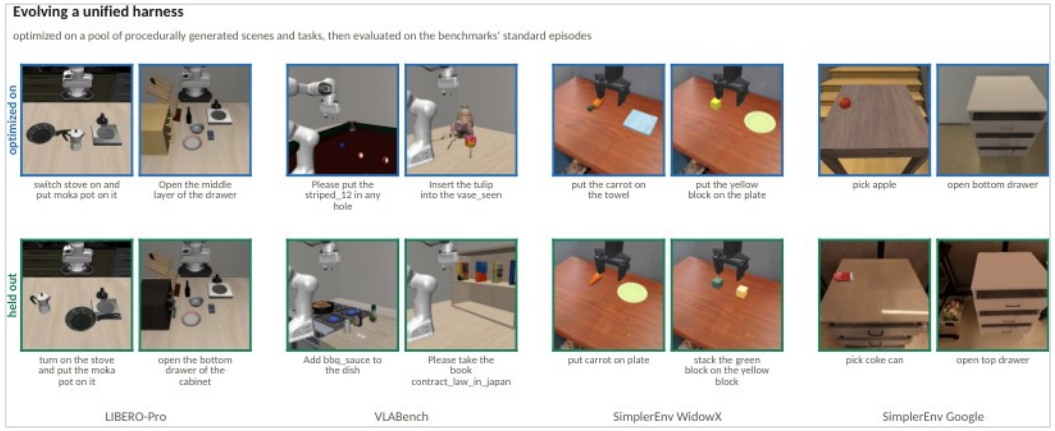
claude-opus-5 reads full rollouts → diagnoses → edits code; accepted only if train score improves; ⩽5 rounds

Persistent memory

Persists across episodes; no preset schema: the coding model decides what to store and how to retrieve

Evaluation: evolve only on training episodes, score everything on held-out ones





① Unseen scenes (left) ② Unseen tasks (right): blue = optimize, green = test

③ Unified harness: 2,201 procedural train → 2,160 standard test

Distance from policy training data to test benchmark (Table 1 colors)

In-distribution
Tuned on this benchmark

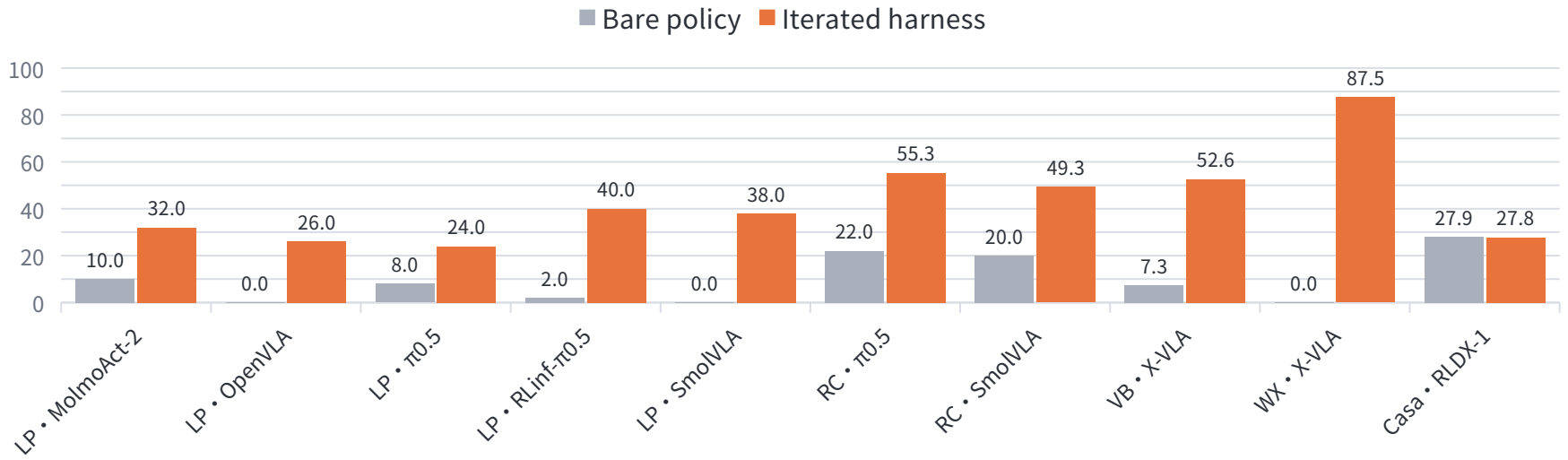
Spatial swap
LIBERO-Pro object positions swapped

Cross-benchmark
Same sim & robot, other benchmark

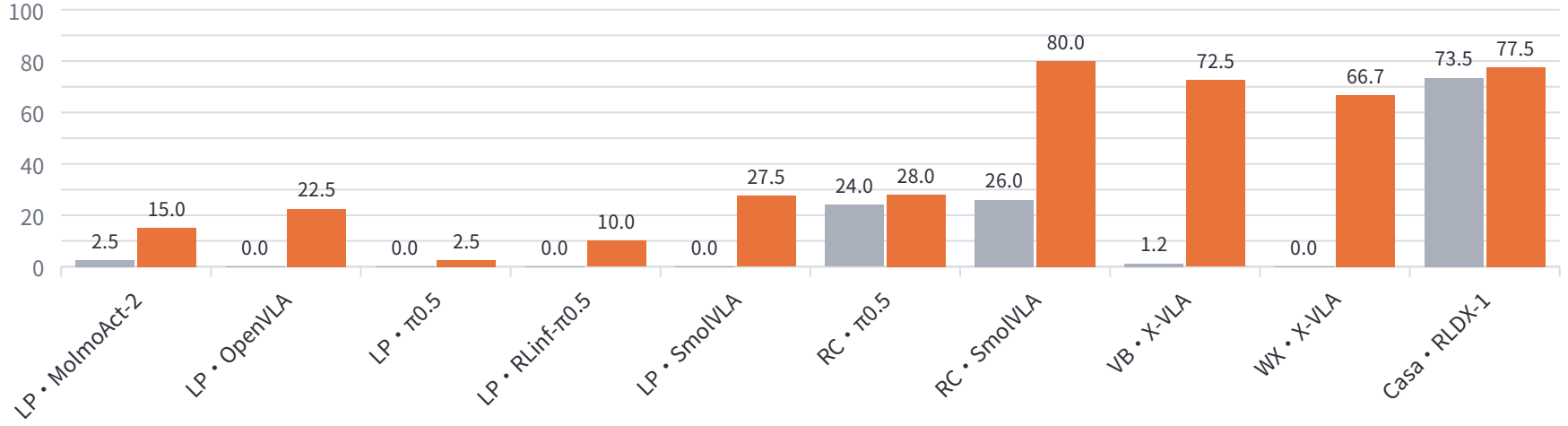
Fully OOD
Generic robot pretraining only

Main results: big gains in 9/10 unseen-scene settings; all unseen-task gains positive

Unseen scenes (%)



Unseen tasks (%)



43.3%

Unseen-scene mean

Bare policy 9.7%

40.2%

Unseen-task mean

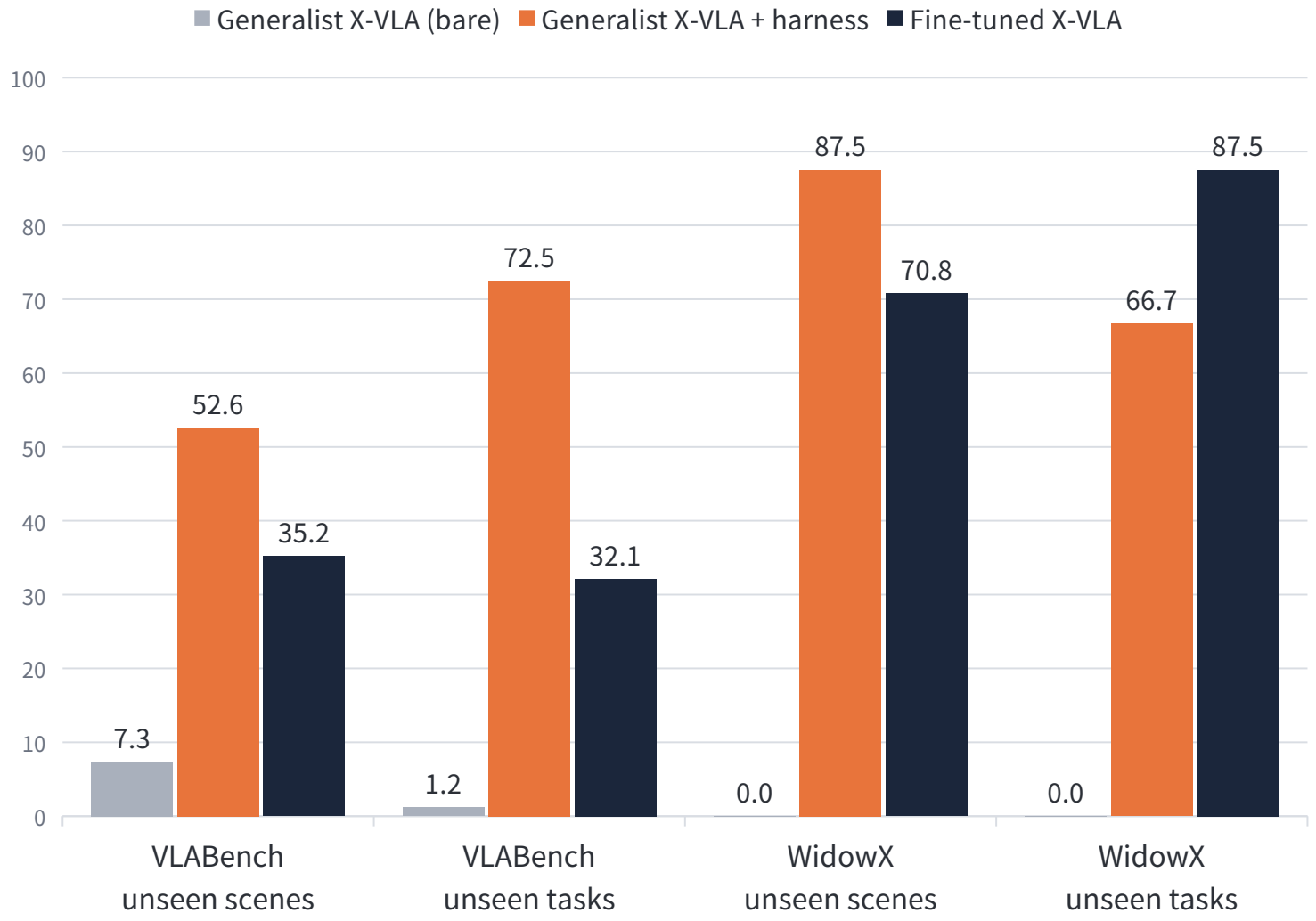
Bare policy 12.7%

Exception

Casa • RLDX-1 (in-dist.) unseen scenes 27.9→27.8; gains only on optimization episodes

arXiv 2610.42141 • RoboArchitect • Source: Table 1 (LP=LIBERO-Pro-10, RC=RoboCerebra, VB=VLABench, WX=WidowX, Casa=RoboCasa-365)

Farther from training data, bigger gains: generalist X-VLA + harness beats fine-tuned in 3/4



3 / 4

comparisons: frozen generalist policy + auto harness beats the same model's fine-tuned weights

Only loss: WidowX unseen tasks 66.7 vs 87.5

By distance to training source (iterated harness)

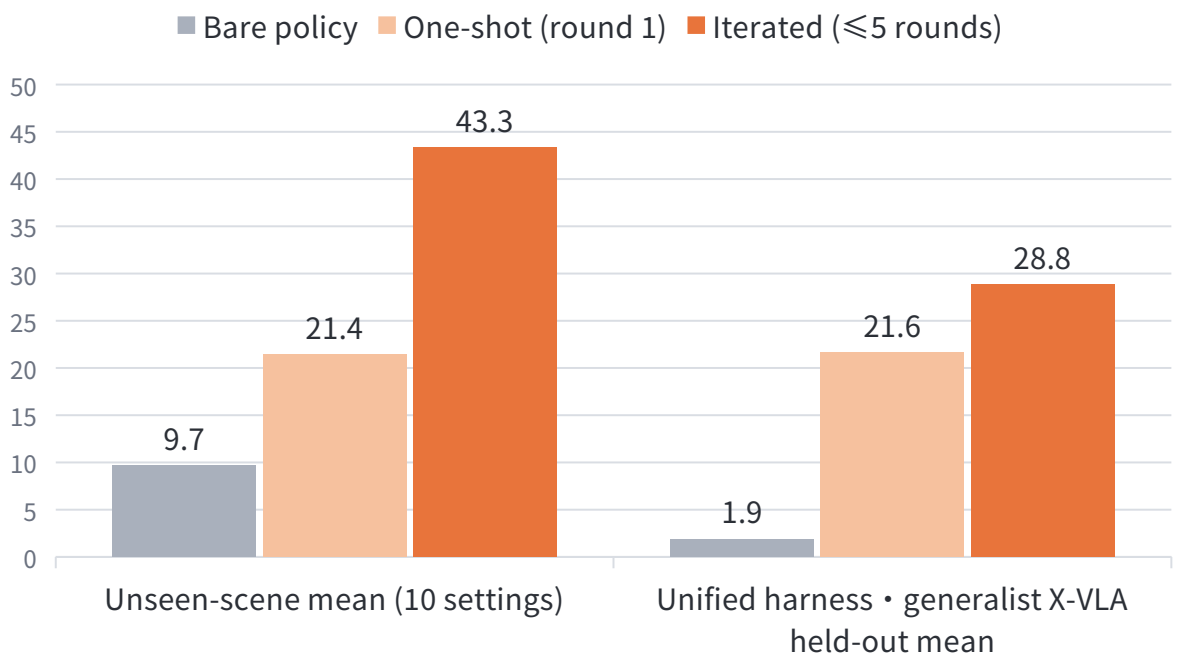
Cross-bench + OOD: **7.3→58.4** (unseen scenes)

Cross-bench + OOD: **6.3→48.7** (unseen tasks)

In-dist. RoboCasa: **27.9→27.8**

Note: fine-tuned baseline covers only X-VLA on 2 benchmarks

Iteration matters: one-shot design gets halfway; iteration reliably beats the bare policy



One-shot can hurt

RC · SmolVLA unseen tasks

26 → 16 → 80

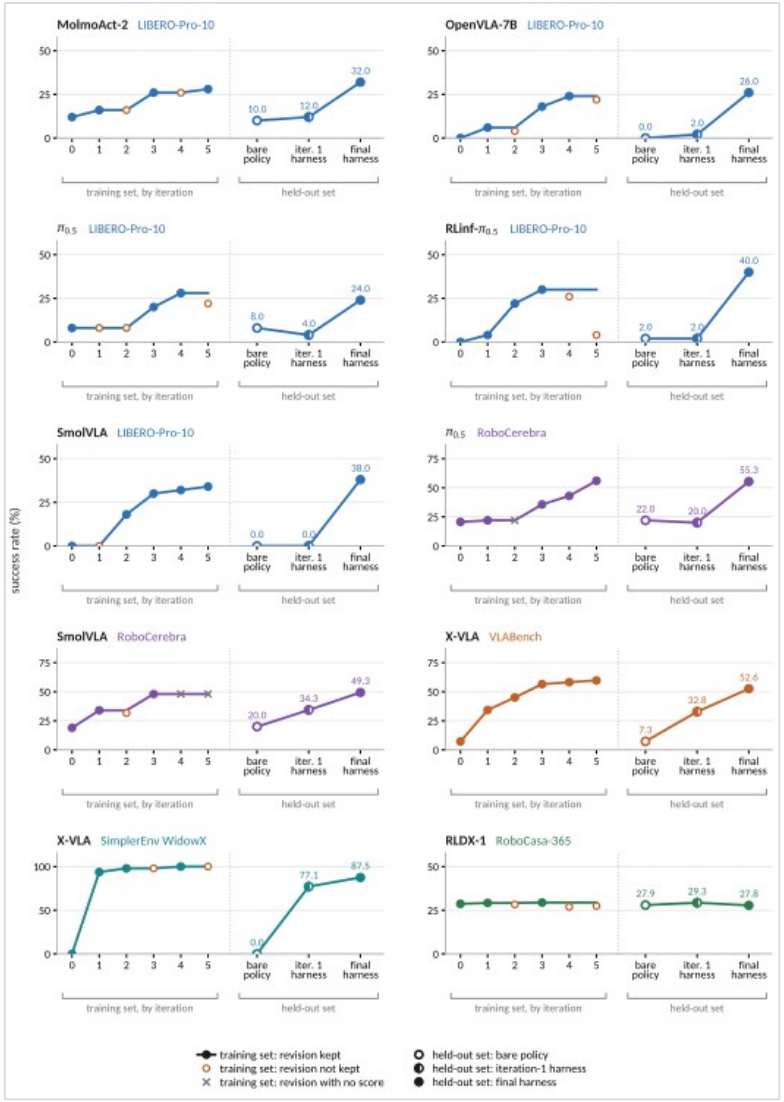
LP · π0.5 unseen scenes 8 → 4 → 24

Counterexample: iteration can overfit

Unified harness · WidowX held-out

45.8 → 24.0

(round 1 → final; not discussed in text)



Rollout evidence alone decides how policy and code divide the work

Code only drops useless policy

Generalist X-VLA keeps gripper open on WidowX → evolves into visual code-as-policy (train 0→100%)



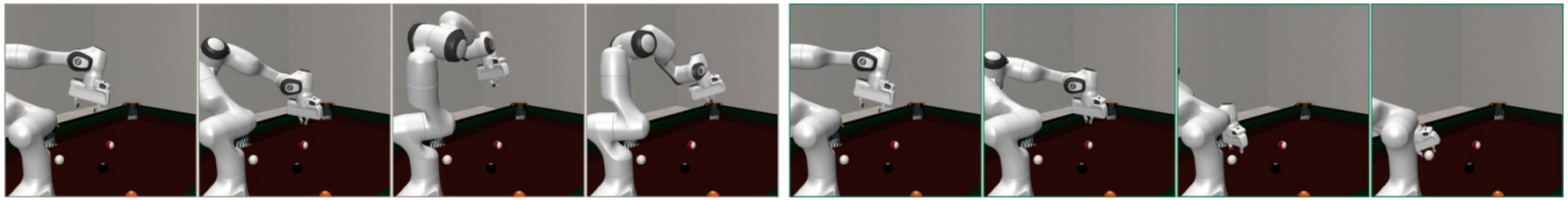
Shared control code fixes decisions

Detects commanded vs. actual motion mismatch (stuck) → lifts arm to free it, then re-queries policy



Policy-led code takes over on failure

Wraps fine-tuned expert: near timeout and still far from target → pointing tool takes over grasp; if approaching, hands back



Evolved control routines: each edit targets a specific failure, as readable code

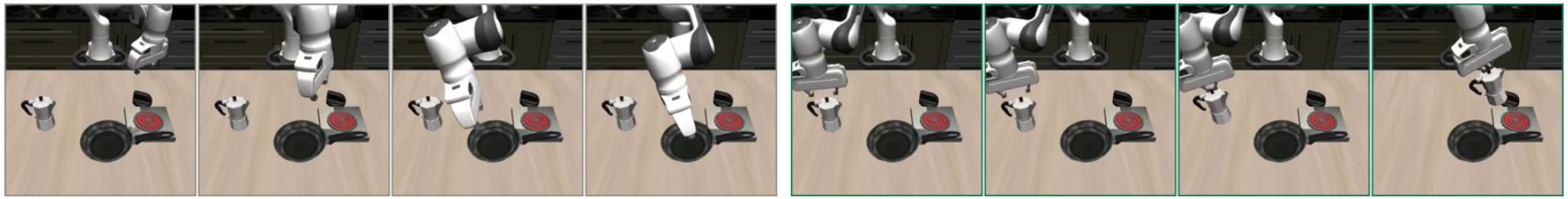
(b) Add motion primitive
Shelf above blocks depth sensing → reach in horizontally for the book (VLABench)



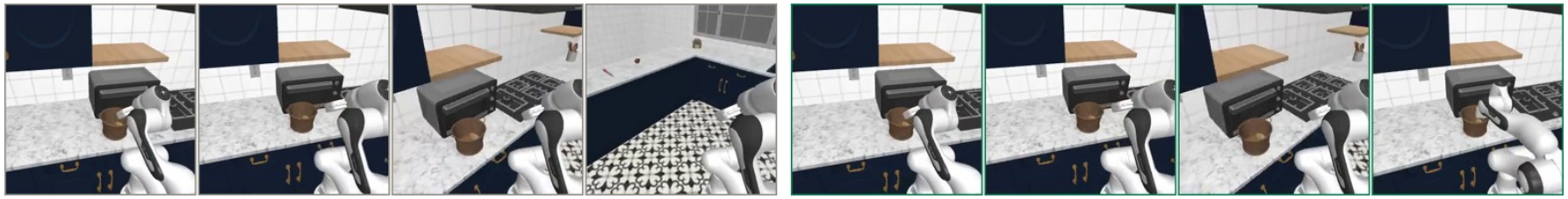
(c) Amplify contact force
Oven door pushed too gently → keep direction, scale up action (RoboCasa)



(d) Fix position memory
Policy reaches for memorized training positions → pointing model finds true target (LIBERO-Pro)



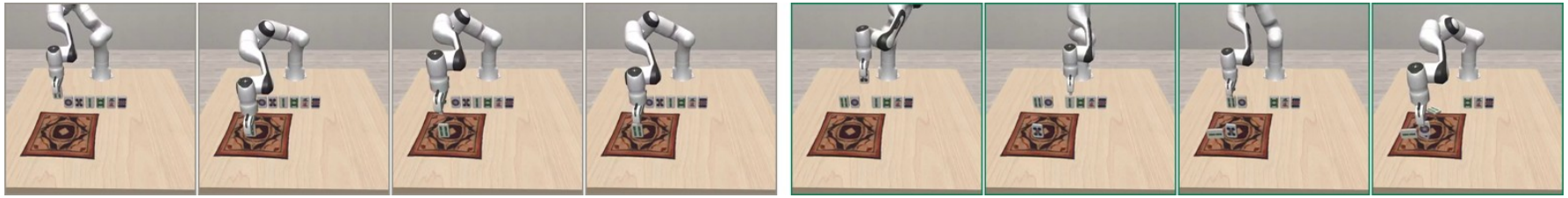
(f) Constrain mobile base
Base drifts off workspace → limit and retract, anchored to gripper action position (RoboCasa)



Self-correction also covers perception and embodiment: work around tool limits, adapt to robot speed

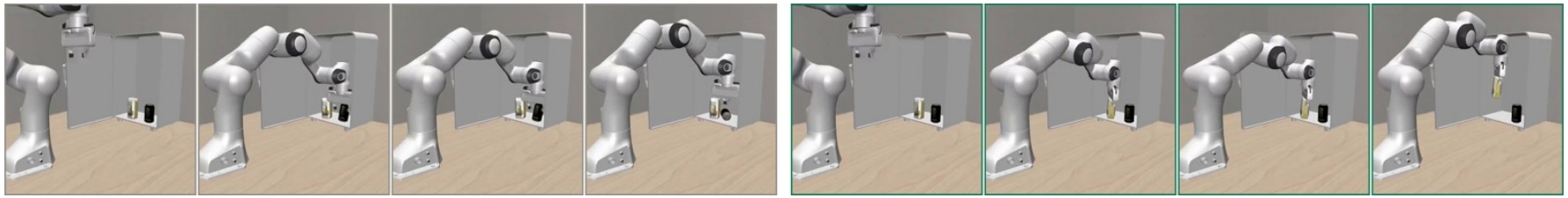
(g) Rank and try candidates

VLM can't tell mahjong tiles apart → keeps a candidate list, tries each until done (VLABench)



(h) Depth-verified grasp

False visual grasp checks cause early release → check for a surface below the fingertips



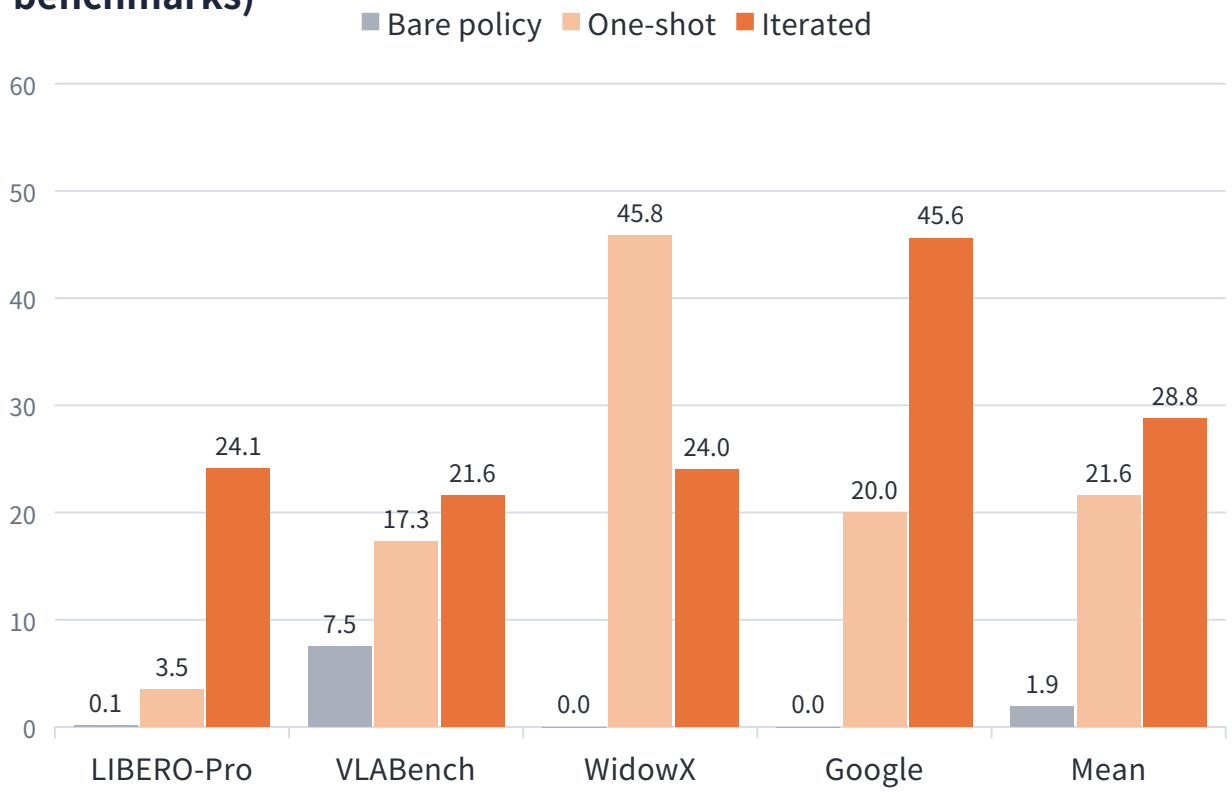
(7c) Arrival, not duration

3 Hz Google robot timed out at fixed duration → end on arrival/stall; train pool 11.0→30.2%

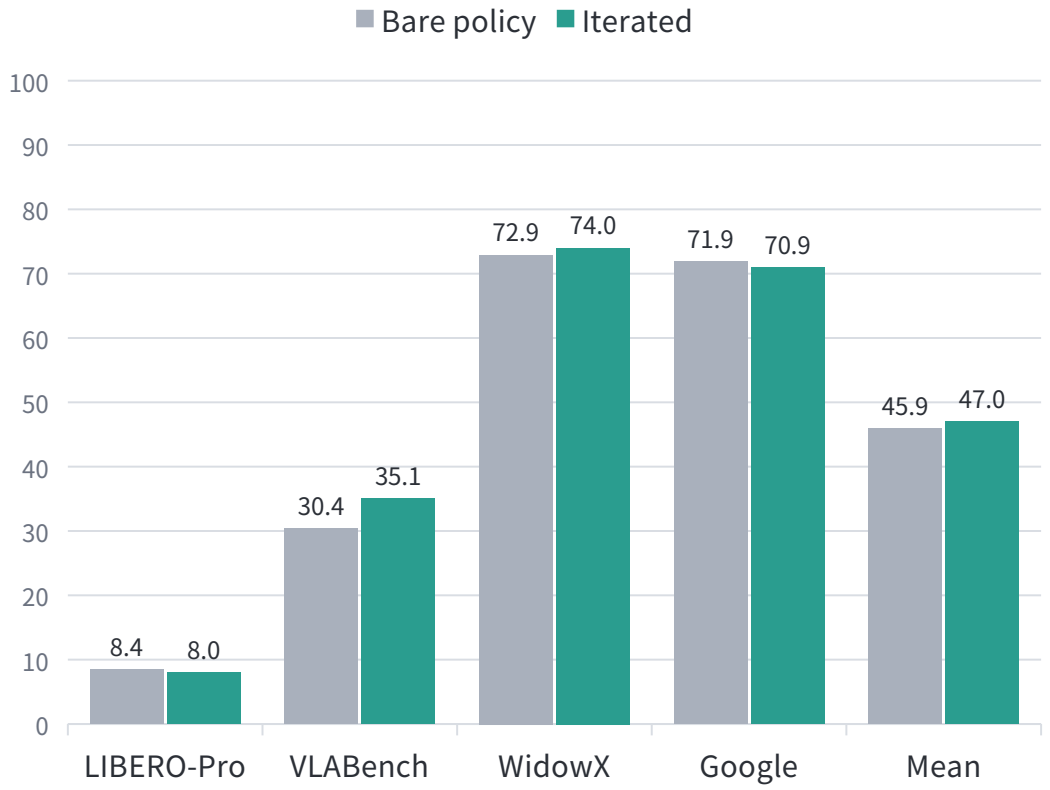


Unified harness: lifts generalist policy from ≈0; experts keep their original level

Wrapping generalist X-VLA (never trained on these benchmarks)



Wrapping per-benchmark fine-tuned X-VLA experts



1.9→28.8%
Generalist held-out mean
Evolves into pure code-as-policy, replacing the VLA

45.9→47.0%
Expert held-out mean
Keeps existing skills, intervenes only on failure

0
Hard-coded embodiment constants
Differences read from action specs and proprioception

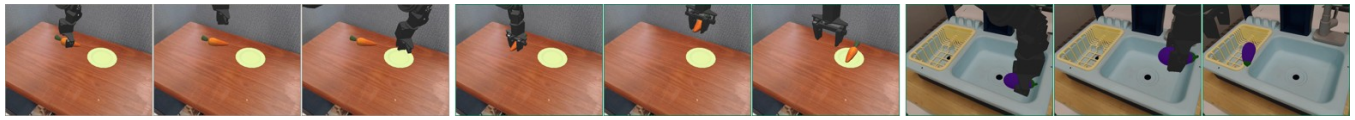
Persistent memory emerges: indexed by physical quantities, >90% held-out read hit rate

Setting	Memory content (designed by the harness)	Reads/ episode	Hit rate
X-VLA • VLABench	Finger depths that lifted objects	3.4	98%
X-VLA • WidowX	Closing height indexed by measured object height	3.5	96%
MolmoAct-2 • LIBERO-Pro	Mean grasp depth below object top	9.9	99%
π0.5 • RoboCerebra	Sim's 150-step scene-reset period	13.0	95%
RLDX-1 • RoboCasa	Which instructions need holding	11.0	92%

Reusable execution experience
Grasp height / depth, pinch or spread grip

Task & environment cues
Holding needed?, reset period, cached LLM parses

Embodiment self-calibration
Opens/closes empty gripper; logs finger motion from depth



(a) WidowX: grasp heights logged by object height transfer to new objects/tasks



(d) Unified harness: gripper calibration on Google robot and Franka

Assessment: strong, readable and auditable idea, but limited statistics and baselines

Strengths

Clear thesis: adapt to new settings by editing code, not weights; evidence spans 5 benchmarks × 7 policies × 2 protocols + cross-embodiment

Unifies two paradigms: data decides the VLA/code mix, reducing naturally to code-as-policy or expert orchestration

Readable & auditable: each edit targets a specific failure mode, easy to inspect and iterate

Low cost: <6 h, <$150, no demos, no training

Sound memory design: self-indexed by measured physical quantities; transfers to new objects

Caveats

Small samples: 10 trials/task (WidowX 24); no confidence intervals or variance across evolution runs

Missing key baselines: no direct comparison with hand-built harnesses (VoLo, etc.) or pure coding agents (CaP-X); fine-tuned baseline only X-VLA on 2 benchmarks

Gains concentrate on weak policies: in-dist. RoboCasa ≈0, experts +2%; multipliers inflated by low baselines

Overfitting risk: edits accepted on train set only; unified harness WidowX 45.8→24.0

Sim-only & single coding model: relies on depth and stable tool outputs; only claude-opus-5, fixed 5 rounds

Conclusion: the headroom lies in the agent harness, which can be designed automatically

1

Headroom is in the harness

Frozen policy, only outer code edited: held-out mean 9.7→43.3% / 12.7→40.2%; larger gains farther from training distribution

2

Design can be automated

Coding model reads rollouts → diagnoses → edits code, self-improving; readable routines and persistent memory emerge, unifiable across embodiments

3

Practical alternative to fine-tuning

No demos, no training, <$150; but "beats fine-tuning" holds in few settings, needing real-robot and stronger-baseline validation

Future directions proposed by the paper

- Saturation point with more iterations
- Contact-rich tasks: dexterous manipulation, tool use
- Sim-to-real transfer of evolved harnesses